Как закрывать агентами весь SDLC

Что нужно, чтобы агент закрывал весь цикл разработки до деплоя в продакшен

Павел Федотовский

AI Engineering Lead в Яндексе

- **Больше 10 лет в .NET**
  на разных инженерных позициях.
- **2 года в AI-кодинге.**
- **Продвигаю AI-инструменты**
  и строю бенчмарки skills.



Роль инженера расширяется от кода до всего цикла

SDLC	Задача	План	Код	Проверка	Деплой	Runtime
РАНЬШЕ	Кто-то ещё		Инженер **пишет код**	Передал дальше		
ТЕПЕРЬ	Инженер отвечает за результат		`агент проходит весь цикл`			

Если роль заканчивается на коде, агент вас заменит.

Для кого этот доклад

01

Начинаете или используете агента только для кода

Поймёте, с чего начать и как провести агента через весь SDLC — от задачи до production.

02

Уже проводите агента через весь SDLC

Сверите свой подход и найдёте идеи, что улучшить.

Где можно применить

Сервис или небольшой набор микросервисов

- Понятный контекст
- Быстрые проверки
- Независимый деплой

Можно и в других системах

Требует больше подготовки: шире контекст, дольше проверки.

План доклада

- Пример
- Контекст и инструменты
- Проверяемый результат
- Роль инженера

ЧАСТЬ 0

Пример

Эксперимент с полным SDLC через агента

Эксперимент

Условия

Создать сервис с нуля

- только через промпты агенту;
- я не знал ни Go, ни внутреннюю инфраструктуру.

Агент сам

- проектирует и пишет код;
- разворачивает сервис и базу;
- настраивает логи и алерты;
- разбирается с проблемами в production.

Агент прошёл путь от шаблона до production

Шаблон Go сервиса

→

Production

Рабочий сервис

API, Postgres, роли, деплой, алерты и мониторинг.

```
# реальная задача агенту

> I've got multiple alerts on PROD,
  please investigate.
```

Затраты первого прохода

ВРЕМЯ

2–3 недели

СТОИМОСТЬ

~$2000

Проблемы первого прохода

≈ 20%

работа над сервисом

≈ 80%

среда вокруг агента

локальный запуск · инфраструктурные ошибки
документация · skills

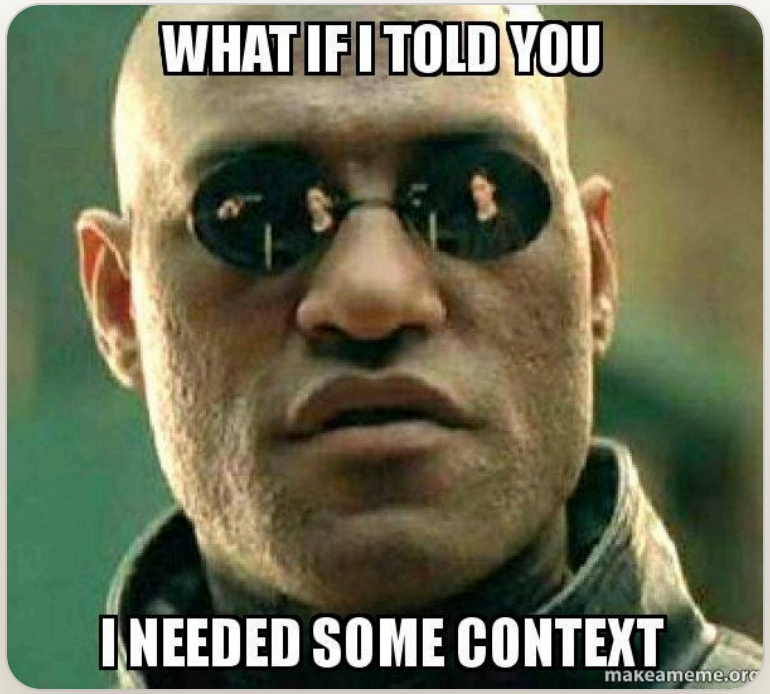
Много вечеров отладки · ≈ 10 задач инфраструктурным командам

ЧАСТЬ 1 ИЗ 3

Контекст и инструменты

Что нужно агенту, чтобы понять задачу и самостоятельно действовать

01 / 03



Всё начинается с контекста

- что это за сервис и как он устроен;
- какие решения уже приняты;
- как его запускать и проверять;
- где искать похожие PR, задачи и обсуждения.

> Без контекста агент начинает с расспросов и угадывания.

AGENTS.md: рабочий контур репозитория

```
# Presentation Workflow
## Required Route
Use $presentation-maintainer for every deck edit.
slides.md = order; pages/*.md = editable sources.
Show Pavel an exact proposal before editing.

## Source Map
current-state.md → narrative and order
presentation-requirements.md → stable rules
todo.md → open feedback
research-notes.md → verified sources

## Boundaries
Preserve work; never create a Git stash proactively.
Keep preview on loopback.
```

Публичные примеры: Codex · Temporal

AGENTS.md: правила конкретного сервиса

```
## Repository facts
Monorepo: Arcadia.
Go via ya make; no go.mod.
VCS: arc. Do not use git or gh.

## Commands
cd cmd/<service> && ya make
ya make -tt <focused-test>

## Validation checklist
1. Build the service.
2. Regenerate .go after proto or OpenAPI changes.
3. Keep configs compatible.
4. Update ya.make for new files.
5. Update docs with behavior or config changes.

## Stable gotchas
Regenerate generated files; never edit.
```

Как завести AGENTS.md

Codex

`/init`

Создаёт `AGENTS.md`.

Cursor

Промпт агенту

«Создай мне `AGENTS.md` по этому проекту».

Claude Code

`/init`

Создаёт `CLAUDE.md`.

Добавьте `@AGENTS.md`.

Проверьте созданный файл: команды и ограничения должны соответствовать проекту.

Контекст хранится рядом с кодом

```
my-service/
├─ AGENTS.md  ← входная точка
├─ ARCHITECTURE.md  ← устройство проекта
├─ CHANGELOG.md  ← что менялось
└─ docs/
   ├─ plans/  ← планы решений
   ├─ local-run.md  ← инструкция
   └─ pitfalls.md  ← подводные камни
```

Контекст за пределами репозитория

ЗНАНИЯ

Вики, тикеты, внутренняя документация

CI

Сборка, тесты, деплой — что сейчас происходит

ОКРУЖЕНИЕ

Логи, алерты, метрики — как сервис живёт

Если агент не может сам получить эти факты, человек передаёт их вручную и становится узким местом.

Агент может пройти весь цикл через браузер

Агент
получает задачу

→

Браузер
кликает интерфейс

→

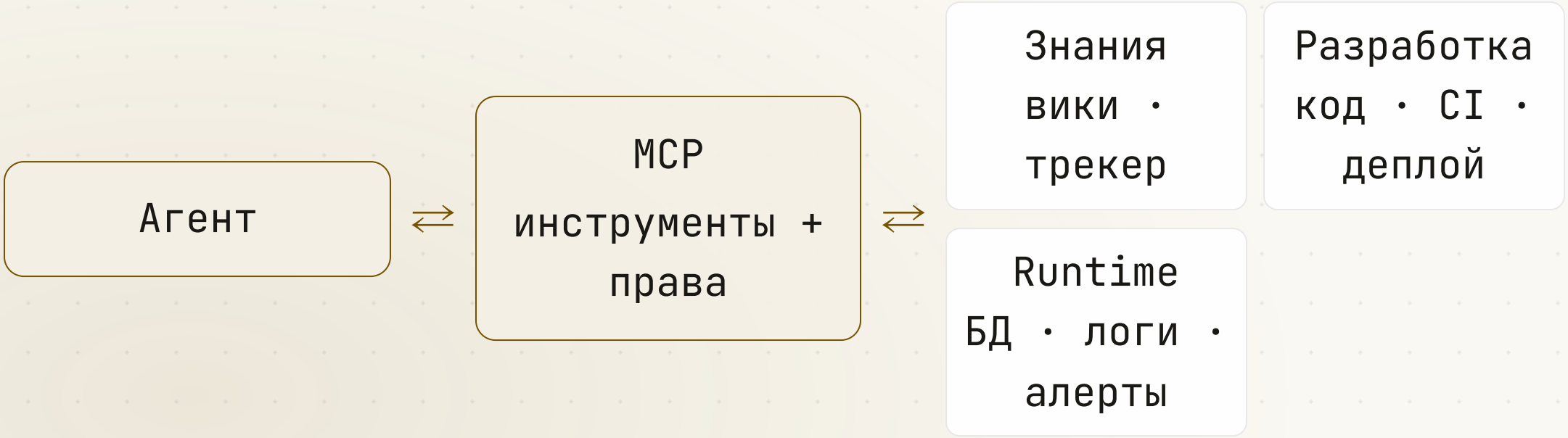
Рабочие системы
тикеты, деплой, логи

МИНУСЫ

- Действия через UI сложнее ограничивать
- Страницы расходуют много токенов

Инструменты: управляемый доступ к сервисам

Model Context Protocol даёт агенту единый управляемый доступ к сервисам.



Что агент может с MCP

01

Прочитать документацию

При необходимости найти похожие тикеты и прочитать логи.

02

Действовать в окружении

Запустить деплой и проверить состояние.

MCP даёт возможность действовать.
Границу задаёт политика вне модели.

Примеры MCP

Разработка

GitHub · GitLab

репозитории · задачи · CI

Данные

PostgreSQL · SQL Server

запросы · схема · операции

Runtime

Grafana · Octopus Deploy

логи · алерты · деплой

Рабочие задачи

TickTick

задачи · списки · планирование

MCP лучше проектировать от сценариев

В идеале: сценарий → узкие операции → API. На практике часто начинают с готового API.

Поверх OpenAPI

Сгенерировать основу

FastMCP · Speakeasy

На .NET

MCP C# SDK · preview

```
dotnet new mcpserver
-n MyMcpServer
```

Проблемы MCP

01

Нет рабочего сценария

Агент видит инструменты, но не знает порядок шагов и критерий результата.

02

Схемы занимают контекст*

Чем больше инструментов, тем дороже выбрать нужный.

* Зависит от клиента: progressive discovery загружает схемы по запросу.

Рабочий сценарий можно оформить в skill

```
web-artifacts-builder/
├── SKILL.md
│   ├── name + description
│   └── пять шагов сборки
└── scripts/
    ├── init-artifact.sh
    └── bundle-artifact.sh
```

anthropics/skills ↗

01 Запрос
«Собери веб-приложение»

02 Выбор skill
по name + description

03 Сценарий
инициализация → код → bundle

Команда .NET уже собрала готовые skills

Сборка

MSBuild · NuGet

Тесты

запуск · фильтры · coverage

Данные

EF Core · доступ к данным

Диагностика

производительность · инциденты

github.com/dotnet/skills ↗

MCP не обязательно подключать к агенту напрямую

```
# посмотреть доступные инструменты
mcporter list github

# прочитать issue через GitHub MCP
mcporter call github.issue_read method=get \
  owner=openai repo=codex issue_number=123
```

mcporter ↗ · mcp2cli ↗

Как skill использует MCP в сценарии

```
Yandex Tracker stores issues in
queues, projects, portfolios,
and goals.

## How To Call MCP
… tracker_mcp --tool <ToolName>
key:value

## Find my open issues
1. Filter: assignee = me(),
status = open
2. Call GetIssues
3. Return key, summary, status
```

01 Описание
с чем работает skill

02 Вызов MCP
как вызвать инструмент

03 Сценарий
фильтр → GetIssues → ответ

Что предоставляют компании: MCP и skills

ИНСТРУМЕНТ	MCP	SKILLS
Grafana	OSS + Cloud	готовый набор
GitLab	встроен в GitLab	для `glab`
Octopus Deploy	официальный сервер	в Agent Step

Как подключить агента к БД

POSTGRESQL

CLI · psql

Skill · Neon Agent Skills

MCP · Postgres MCP Pro

SQL SERVER · .NET

CLI · sqlcmd

Skill · EF Core

MCP · Microsoft SQL MCP

Testing — агент работает сам.

С чего начать?

Skill over API

Если прямой вызов разрешён

`SKILL.md` описывает API или CLI и пользовательский сценарий.

Skill over MCP

Если нужны права и аудит

MCP управляет доступом, skill задаёт порядок действий.

Начинайте со Skill over API. Переходите к Skill over MCP, когда этого требуют безопасность или аудит.

MCP IS DEAD?


kitze 🛠️ tinkerer.club
@thekitze
this was just glorious to witness over the last year
mcp
cli
7
391
February 24th, 2026 at 11:50 PM

Нет. MCP продолжает развиваться

01 · В работе

Skills over MCP ↗

Может убрать отдельные skills-обёртки над MCP.

02 · 28 июля

Stateless и sessionless

MCPorter 0.13.3 ↗ уже поддерживает новый и legacy flow.

03 · Эксперимент

Tasks ↗

Долгие операции с отслеживаемым состоянием.

Как собрать harness для команды?

Skills
сценарии сервисов

+

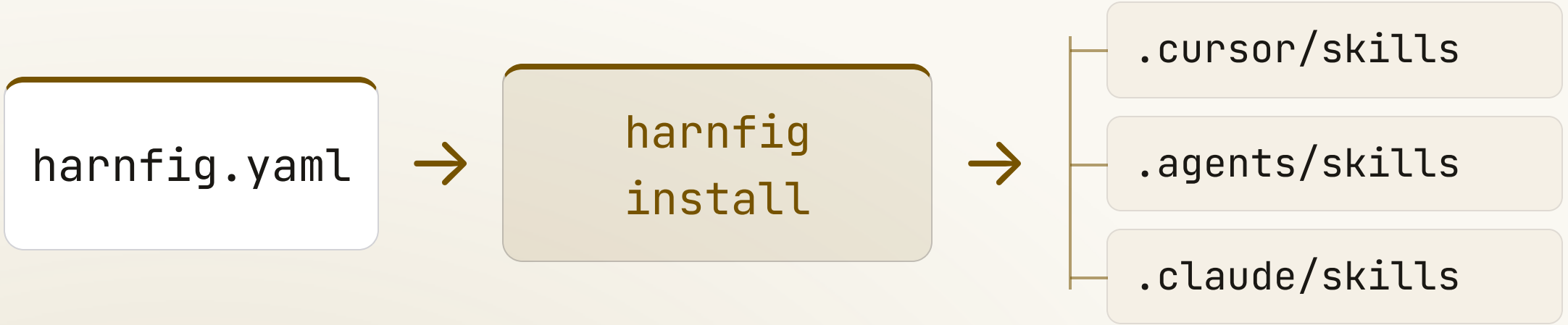
MCP
доступ к сервисам

→

Шаблон проекта
готов для команды

Нужен один способ подключить инструменты своих сервисов.

Как подключить skills к разным агентам



В Яндексе — `aisuite` · open source — harnfig ↗

harnfig.yaml: что установить и куда

```yaml
version: 1
scope: project
agents:
  - cursor
  - codex
  - claude
skills:
  - repo: mattpocock/skills
    path: skills/productivity/grill-
me
    ref: main
```

agents

Куда поставить

repo + path

Откуда взять skill

ref

Ветка, тег или commit

ЧАСТЬ 2 ИЗ 3

Проверяемый результат

Контекст и инструменты дают возможность действовать.
Автономность начинается, когда агент сам видит результат

02 / 03

Проверка должна быть рядом с изменением

Чем позже обратная связь, тем дороже ошибка.

- **Тесты**
  проверяют поведение кода;
- **Локальный запуск**
  — работу сервиса;
- **Логи и алерты**
  — результат после релиза.

Локальный запуск — тоже продукт

ПРОВЕРКА ЧЕРЕЗ TESTING

деплой → ожидание → ошибка

≈ 15 минут

ЛОКАЛЬНАЯ ПРОВЕРКА

одна команда + тестовые данные

запуск → `healthcheck` → `smoke-test`

≈ 1 минута

Хорошая среда разработки = хорошая среда для агента.

Настройка локальной проверки требует времени

01

Настроить

Команда запуска, локальные зависимости, БД и тестовые данные.

02

Описать

Docs, skill и однозначный критерий успешной проверки.

03

Поддерживать

Обновлять вместе с сервисом и сохранять рабочий smoke-test.

Инфраструктура должна быть доступна агенту

ВАРИАНТ	ВХОД ДЛЯ АГЕНТА	КОГДА ПОДХОДИТ
OpenTofu	CLI + skill	обычный IaC
Pulumi	MCP	привычные языки
Crossplane	Kubernetes API + skill	уже есть Kubernetes

Без инфраструктуры как кода агент будет останавливаться.

После релиза агент проверяет результат по логам и алертам

Алерт

«Разберись и почини»

→

Причина

Прочитал логи и связал алерт с изменением

→

Исправление

Исправил и дождался, пока алерт погас

Первый проход покажет, что внедрять

01

Нет доступа к инфраструктуре

Подключите нужный сервис через skill или MCP.

02

Нет локального запуска

Добавьте команду запуска и быструю проверку.

03

Не хватило контекста

Запишите знания в память проекта.

Место остановки агента — следующий шаг внедрения.

SDLC агента — цикл с обратной связью



После каждого прохода переносите проверенные знания в контекст агента.

ЧАСТЬ 3 ИЗ 3

Роль инженера

Когда контекст, инструменты и проверки собраны — что делает человек

03 / 03

Проблема: работа без плана

СОБЛАЗН

задача → сразу писать код

РЕЗУЛЬТАТ

реализация → агент ушёл не туда

отладка → удаление → новая попытка

Попытка сэкономить на плане заканчивалась дополнительной работой.

Сначала план, потом код

УТОЧНЕНИЕ ПЛАНА

4–5

итераций до реализации

- агент собирает контекст и пишет план;
- инженер задаёт уточняющие вопросы;
- `docs/*.md` хранит итоговый план;
- агент реализует, проверяет и готовит пулреквест.

Дешевле поправить план до изменений, чем разбирать большой неправильный набор правок.

Автоматизируйте обязательные проверки

Если шаг обязателен, он должен запускаться автоматически.

ПОСЛЕ ИЗМЕНЕНИЯ

format · lint

быстрые тесты

ПЕРЕД MERGE · CI

полный набор тестов

security scan · docs check

Проблема: синхронизация документации

ДО HOOK

Просил обновлять docs вместе с кодом

агент мог пропустить инструкцию

документацию проверял вручную

```
// .claude/settings.json → hooks
"Stop": [{
  "hooks": [{
    "type": "prompt",
    "prompt": "Check docs against
      changed code: $ARGUMENTS"
  }]
}]
```

Инженер проектирует систему уровнем выше кода

КЛАССИЧЕСКИЙ ЦИКЛ

Инженер ведёт каждый шаг

задача → код → тест → деплой → логи

Агент помогает только с кодом.



АГЕНТНЫЙ ЦИКЛ

Агент проходит весь путь

план → изменение → проверка → runtime

Инженер: требования, архитектура и приёмка.

Визуализируйте, что происходит в системе

Админка

состояние базы · безопасные операции

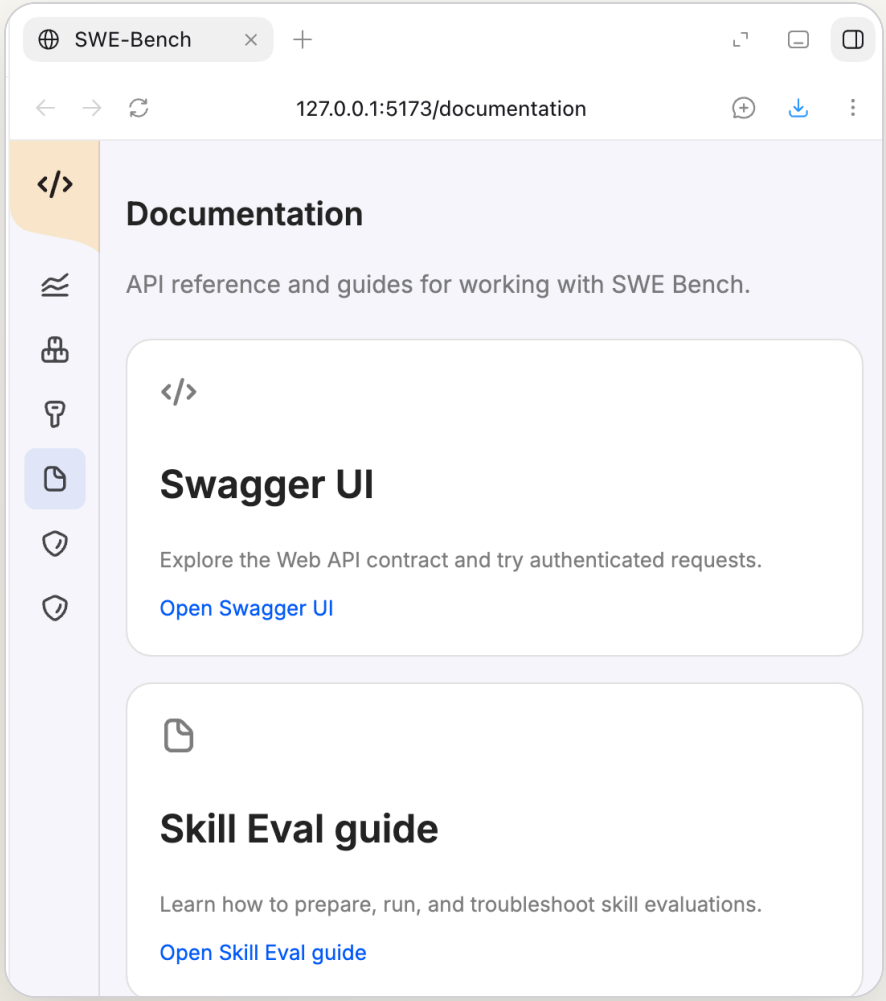
Архитектура сервиса

компоненты · зависимости · потоки данных

Страница с полезными ссылками

Swagger UI · документация · инструкции

Агент может собрать такие страницы прямо внутри приложения.



Что может сделать каждый

	КТО	ЧТО МОЖЕТ СДЕЛАТЬ	МАСШТАБ
01	Разработчик	Настраивает локальный запуск и записывает знания о проекте.	сам
02	Команда	Автоматизирует приёмку перед слиянием и выкладкой.	в проекте
03	Платформа	Даёт агенту доступ к сервисам и помогает переиспользовать skills.	в компании

Что забрать с собой

- Агент закрывает SDLC, когда у него есть **контекст, инструменты и проверки.**
- Инженер **задаёт границы** и принимает результат.

Хорошая среда разработки
= хорошая среда для агента.

harnfig ↗
Установить и зафиксировать skills.

dotnet/skills ↗
Взять готовые skills для .NET.